```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import datetime as dt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import seaborn as sns

df=pd.read_csv("C:/Users/User/Downloads/sales_data.csv",
encoding='unicode_escape')

df.head()
```

```
   ORDERNUMBER  QUANTITYORDERED  PRICEEACH  ORDERLINENUMBER
SALES  \
0        10107               30      95.70                2  2871.00

1        10121               34      81.35                5  2765.90

2        10134               41      94.74                2  3884.34

3        10145               45      83.26                6  3746.70

4        10159               49     100.00               14  5205.27


         ORDERDATE   STATUS  QTR_ID  MONTH_ID  YEAR_ID  ...  \
0   2/24/2003 0:00  Shipped       1         2     2003  ...
1    5/7/2003 0:00  Shipped       2         5     2003  ...
2    7/1/2003 0:00  Shipped       3         7     2003  ...
3   8/25/2003 0:00  Shipped       3         8     2003  ...
4  10/10/2003 0:00  Shipped       4        10     2003  ...

                     ADDRESSLINE1  ADDRESSLINE2           CITY STATE  \
0        897 Long Airport Avenue           NaN            NYC    NY
1             59 rue de l'Abbaye           NaN          Reims   NaN
2  27 rue du Colonel Pierre Avia           NaN          Paris   NaN
3             78934 Hillside Dr.           NaN       Pasadena    CA
4                7734 Strong St.           NaN  San Francisco    CA

  POSTALCODE COUNTRY TERRITORY CONTACTLASTNAME CONTACTFIRSTNAME
DEALSIZE
0      10022     USA       NaN              Yu             Kwai
Small
1      51100  France      EMEA         Henriot             Paul
Small
2      75508  France      EMEA        Da Cunha            Daniel
Medium
3      90003     USA       NaN           Young             Julie
Medium
4        NaN     USA       NaN           Brown             Julie
```

```
Medium

[5 rows x 25 columns]

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2823 entries, 0 to 2822
Data columns (total 25 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   ORDERNUMBER       2823 non-null   int64
 1   QUANTITYORDERED   2823 non-null   int64
 2   PRICEEACH         2823 non-null   float64
 3   ORDERLINENUMBER   2823 non-null   int64
 4   SALES             2823 non-null   float64
 5   ORDERDATE         2823 non-null   object
 6   STATUS            2823 non-null   object
 7   QTR_ID            2823 non-null   int64
 8   MONTH_ID          2823 non-null   int64
 9   YEAR_ID           2823 non-null   int64
 10  PRODUCTLINE       2823 non-null   object
 11  MSRP              2823 non-null   int64
 12  PRODUCTCODE       2823 non-null   object
 13  CUSTOMERNAME      2823 non-null   object
 14  PHONE             2823 non-null   object
 15  ADDRESSLINE1      2823 non-null   object
 16  ADDRESSLINE2      302 non-null    object
 17  CITY              2823 non-null   object
 18  STATE             1337 non-null   object
 19  POSTALCODE        2747 non-null   object
 20  COUNTRY           2823 non-null   object
 21  TERRITORY         1749 non-null   object
 22  CONTACTLASTNAME   2823 non-null   object
 23  CONTACTFIRSTNAME  2823 non-null   object
 24  DEALSIZE          2823 non-null   object
dtypes: float64(2), int64(7), object(16)
memory usage: 551.5+ KB

df.columns

Index(['ORDERNUMBER', 'QUANTITYORDERED', 'PRICEEACH',
'ORDERLINENUMBER',
       'SALES', 'ORDERDATE', 'STATUS', 'QTR_ID', 'MONTH_ID',
'YEAR_ID',
       'PRODUCTLINE', 'MSRP', 'PRODUCTCODE', 'CUSTOMERNAME', 'PHONE',
       'ADDRESSLINE1', 'ADDRESSLINE2', 'CITY', 'STATE', 'POSTALCODE',
       'COUNTRY', 'TERRITORY', 'CONTACTLASTNAME', 'CONTACTFIRSTNAME',
       'DEALSIZE'],
      dtype='object')
```

```
to_drop=['ADDRESSLINE1', 'ADDRESSLINE2', 'STATE',
'POSTALCODE','PHONE']
df=df.drop(to_drop, axis=1)

df.isnull().sum()
```

```
ORDERNUMBER            0
QUANTITYORDERED        0
PRICEEACH              0
ORDERLINENUMBER        0
SALES                  0
ORDERDATE              0
STATUS                 0
QTR_ID                 0
MONTH_ID               0
YEAR_ID                0
PRODUCTLINE            0
MSRP                   0
PRODUCTCODE            0
CUSTOMERNAME           0
CITY                   0
COUNTRY                0
TERRITORY           1074
CONTACTLASTNAME        0
CONTACTFIRSTNAME       0
DEALSIZE               0
dtype: int64
```

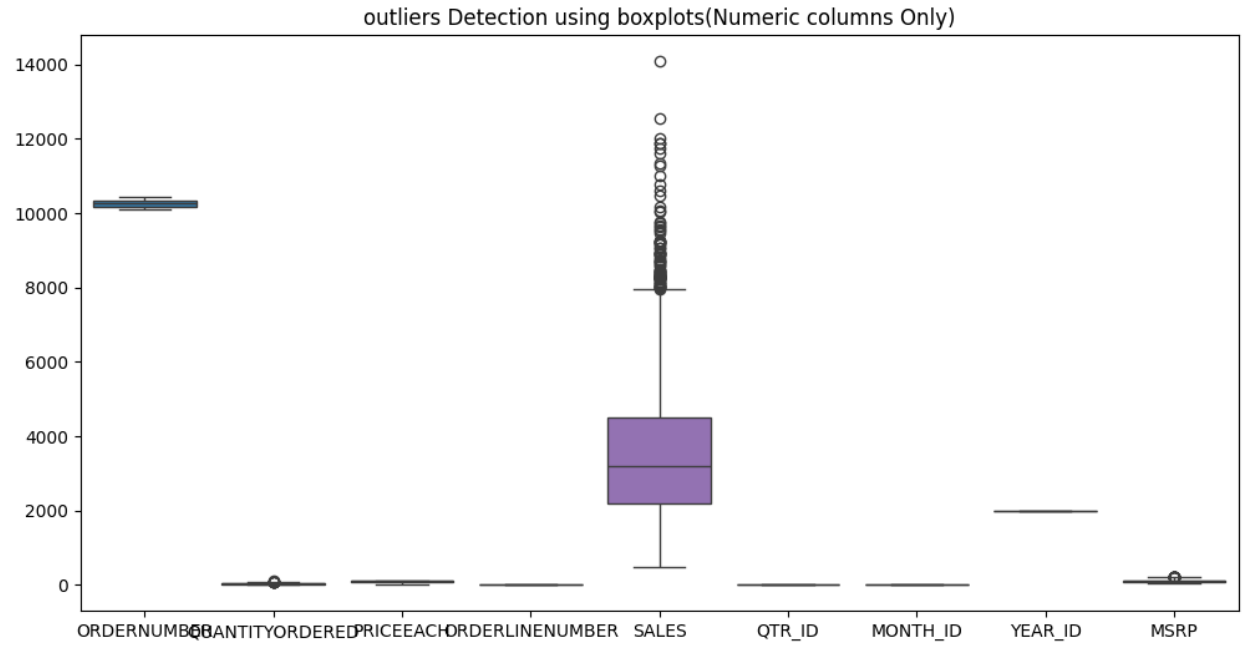
```
df.dtypes
```

```
ORDERNUMBER           int64
QUANTITYORDERED       int64
PRICEEACH           float64
ORDERLINENUMBER       int64
SALES               float64
ORDERDATE            object
STATUS               object
QTR_ID                int64
MONTH_ID              int64
YEAR_ID               int64
PRODUCTLINE          object
MSRP                  int64
PRODUCTCODE          object
CUSTOMERNAME         object
CITY                 object
COUNTRY              object
TERRITORY            object
CONTACTLASTNAME      object
CONTACTFIRSTNAME     object
```

```
DEALSIZE             object
dtype: object

df_numeric=df.select_dtypes(include=['int64','float64'])
import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(12,6))
sns.boxplot(data=df_numeric)
plt.title("outliers Detection using boxplots(Numeric columns Only)")
plt.show()

Q1= df_numeric.quantile(0.25)
Q3= df_numeric.quantile(0.75)
IQR=Q3-Q1
outliers=((df_numeric<(Q1-1.5*IQR)) | (df_numeric>(Q3+1.5*IQR))).sum
print("\n Number of outliers per Numeric Feature:\n",outliers)
```




```
 Number of outliers per Numeric Feature:
 <bound method DataFrame.sum of       ORDERNUMBER  QUANTITYORDERED
PRICEEACH  ORDERLINENUMBER  SALES  QTR_ID  \
0           False            False      False           False  False
False
1           False            False      False           False  False
False
2           False            False      False           False  False
False
3           False            False      False           False  False
False
```

```
4           False             False      False             False  False
False
...           ...               ...        ...               ...    ...
...
2818        False             False      False             False  False
False
2819        False             False      False             False  False
False
2820        False             False      False             False  False
False
2821        False             False      False             False  False
False
2822        False             False      False             False  False
False

      MONTH_ID  YEAR_ID   MSRP
0        False    False  False
1        False    False  False
2        False    False  False
3        False    False  False
4        False    False  False
...        ...      ...    ...
2818     False    False  False
2819     False    False  False
2820     False    False  False
2821     False    False  False
2822     False    False  False

[2823 rows x 9 columns]>
```

```
scaler=StandardScaler()
X_scaled=scaler.fit_transform(df_numeric)
print("Data normalized using StandardScaler.")
```

```
Data normalized using StandardScaler.
```

```
df_normalized=pd.DataFrame(X_scaled, columns=df_numeric.columns)
print("\n Sample of Normalized Data:")
display(df_normalized.head())
print("\n Mean of each feature after normalization :\n",
df_normalized.mean())
print("\n Standard deviation of each feature after normalization:\n",
df_normalized.std())
```

```
 Sample of Normalized Data:

   ORDERNUMBER  QUANTITYORDERED  PRICEEACH  ORDERLINENUMBER     SALES
\
0    -1.647947        -0.522891   0.596978        -1.057059 -0.370825
```

```
1    -1.495888         -0.112201  -0.114450         -0.347015 -0.427897

2    -1.354689          0.606505   0.549384         -1.057059  0.179443

3    -1.235214          1.017195  -0.019759         -0.110334  0.104701

4    -1.083154          1.427884   0.810158          1.783116  0.896740

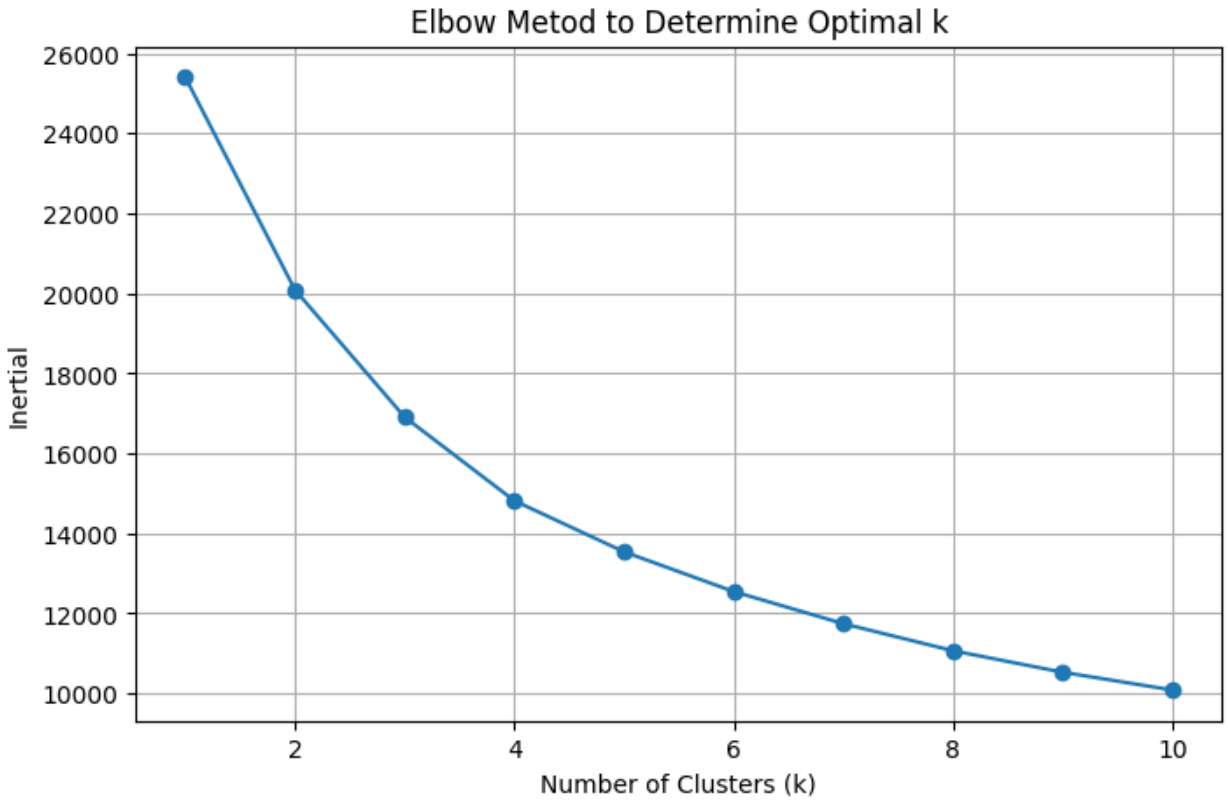

     QTR_ID  MONTH_ID  YEAR_ID      MSRP
0 -1.427039 -1.392909 -1.16517 -0.142246
1 -0.596243 -0.572337 -1.16517 -0.142246
2  0.234553 -0.025289 -1.16517 -0.142246
3  0.234553  0.248235 -1.16517 -0.142246
4  1.065350  0.795284 -1.16517 -0.142246


 Mean of each feature after normalization :
 ORDERNUMBER       -5.927482e-15
QUANTITYORDERED    3.347580e-16
PRICEEACH         -3.221731e-16
ORDERLINENUMBER    9.312817e-17
SALES              1.812224e-16
QTR_ID             7.550932e-17
MONTH_ID          -1.761884e-17
YEAR_ID           -1.988412e-15
MSRP               8.054328e-17
dtype: float64

 Standard deviation of each feature after normalization:
 ORDERNUMBER        1.000177
QUANTITYORDERED    1.000177
PRICEEACH          1.000177
ORDERLINENUMBER    1.000177
SALES              1.000177
QTR_ID             1.000177
MONTH_ID           1.000177
YEAR_ID            1.000177
MSRP               1.000177
dtype: float64
```

```
inertia=[]
K = range(1,11)
for k in K :
    kmeans =KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)
plt.figure(figsize=(8,5))
plt.plot(K,inertia,marker='o')
plt.title("Elbow Metod to Determine Optimal k")
```

```
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Inertial")
plt.grid(True)
plt.show()
```




```
from sklearn.metrics import silhouette_score

for k in range (2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_scaled)
    sil_score = silhouette_score(X_scaled, labels)
    print(f"K = {k} → Silhouette Score = {sil_score:.4f}")
```

```
K = 2 → Silhouette Score = 0.2169
K = 3 → Silhouette Score = 0.1960
K = 4 → Silhouette Score = 0.2059
K = 5 → Silhouette Score = 0.1872
K = 6 → Silhouette Score = 0.1949
K = 7 → Silhouette Score = 0.1922
K = 8 → Silhouette Score = 0.1837
K = 9 → Silhouette Score = 0.1832
K = 10 → Silhouette Score = 0.1842
```

```
#Visualize Clusters for K = 3
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = kmeans.fit_predict(X_scaled)

plt.figure(figsize=(8,6))
sns.scatterplot(
    x=df_numeric['SALES'],
    y=df_numeric['MSRP'],
    hue=labels,
    palette='Set2'
)
plt.title("K-Means Clustering Visualization (K = 3)")
plt.xlabel("Sales")
plt.ylabel("MSRP")
plt.legend(title='Cluster')
plt.show()
```

